

AgentFlow – No-Code Agent Framework Based on Logical Primitives

Dmitry Lande¹, Leonard Strashnoy²

¹ ORCID: 0000-0003-3945-1178

National Technical University of Ukraine – Igor Sikorsky Kyiv Polytechnic Institute

² ORCID: 0009-0008-5575-0286

University of California, Los Angeles (UCLA)

Abstract. This article presents the AgentFlow framework – a new paradigm of no-code programming built on natural-language structural pseudocode, executed through Large Language Models (LLMs). The framework utilizes logical primitives – Condition, Loop, Function, Label, Goto — as fundamental building blocks for constructing complex agent behaviors. An agent model is introduced, where each agent has: a role, state, logic, and communication capability. The concept of a swarm of agents is proposed, enabling LLMs to simulate parallel task execution and process complex queries without writing code. The article also describes a memory system for self-learning, provides examples of data analysis, and outlines formal rules for agent-to-agent interaction.

Keywords: no-code programming, large language models (LLM), agent model, swarm of agents, logical primitives, parallel execution via prompts, agent self-learning

Introduction

No-code programming – that is, programming without code or traditional systems such as Python, C#, or R – enables information analysis through natural language, without requiring technical expertise. AgentFlow is our new approach to no-code programming, implemented via structured pseudocode in natural language. In this framework, the role of a programming system and interpreter is performed by an LLM (large language model), and instead of actual code, logical primitives are used: Condition (If-Else), Loop (For), Function, Label, Goto.

This framework can be applied to data analysis, text processing, forecasting, trend detection, and virtually any task that can be described using logical language. It allows transforming ordinary queries into structured prompts; building swarms of agents that solve tasks simultaneously or sequentially; simulating parallel execution through structured text; introducing internal agent memory for learning and adaptation; and producing structured output suitable for further use.

No-code programming is a paradigm that enables functionality creation without writing code, using visual interfaces or natural language [1]. The framework described in [2] proposes a structured approach to behavior construction through primitives: Condition, Loop, Function, Label, Goto. These elements form the basis for logical process control, similar to classical programming constructs, but expressed via natural-language pseudocode.

Such an approach can be applied both for educational purposes and in real-world analysis systems where users require high adaptability for non-technical individuals.

Agent-based modeling has long been used in artificial intelligence [3–6], particularly in tasks involving prediction, data processing, and automated decision-making.

A classical agent system includes properties such as each agent having its own role, internal state, sensors (input), actuators (output), and communication with other agents.

AgentFlow introduces a new version of the agent model, where agents are created through natural language, without the need for technical implementation. Interaction occurs via structured prompts, and execution is handled by an LLM acting as a programming environment.

Although LLMs have a sequential architecture, by properly structuring prompts it is possible to simulate parallel task execution [7]. This is achieved by explicitly defining each execution branch, treating individual parts as separate entities, and combining results at a single final node – the moderator.

In AgentFlow, this principle is implemented through structured pseudocode, allowing precise control over logic and generating clear, structured output.

Memory systems in agent models allow past tasks to be stored, errors analyzed, and performance improved over time. In Machine Learning terms, this corresponds to reward/punishment systems, where an agent’s state is formed based on past interactions [8].

AgentFlow introduces internal agent memory [9] that stores past tasks, generates accuracy statistics, and is used for behavioral adaptation. This represents a step toward agent self-organization – where agents not only execute tasks but also learn from experience.

Methodology

Basic Primitives of No-Code Programming

AgentFlow is based on five fundamental logical primitives that operate through structured text:

Primitive 1: Condition (Condition, If-Else)

Formalization:

- *Input* – input data (e.g., text, numeric parameter, etc.);
- *C* – condition (predicate) that returns a True or False value;
- *A₁, A₂* – two possible actions executed when the condition is True or False, respectively.

The prompt function *P* is defined as:

$$P(\text{Input}) = \begin{cases} A_1, & \text{if } C(\text{Input}) = \text{True}, \\ A_2, & \text{if } C(\text{Input}) = \text{False}. \end{cases}$$

This formal definition is analogous to the classical *if-else* statement in programming languages, but it is applied through textual instructions.

Primitive 2: Loop

Let:

- *S* = {*s₁, s₂, ..., s_n*} – a set of elements to be processed;
- *F* – an operation applied to each element of the set.

The prompt function *P(S)* is defined as the union of results from applying the operation *F* to each element of the set:

$$P(S) = \bigcup F(s_i).$$

This formal definition is analogous to the classical for loop in programming languages, where the same operation is repeated for all elements of a set.

Primitive 3: Function

Let:

- *F: X → Y* – a function that maps elements from set *X* to elements of set *Y*;
- *x* – input element;
- *parametr* – a set of parameters that control the behavior of the function.

The prompt function *F_{extract}(x, parametr)* is implemented through an instruction passed to the model:

$$F_{extract}(x, parametr) = Promt(x, parametr).$$

This formal definition is analogous to the function abstraction in programming languages, where a function can accept parameters to adjust its internal logic.

The mechanism of the Function primitive involves passing a specific task with parameters to the LLM, defining how it should be processed. This allows for the creation of flexible instructions that can be reused across different inputs.

Primitive 4: Label

Let:

- L – label name;
- $Block$ – a block of instructions.

Formal definition:

$$Label(L, Block) ::= (L: Block)$$

This construct allows explicitly defining the initial state of the system.

Primitive 5: Goto

Let:

- L – a label name for the jump;
- $Condition$ – an optional condition.

Formal definition:

$$Goto(L, Condition) ::= \text{if } (Condition) \text{ then goto } L$$

This enables jumps in the logical structure, which is especially useful for handling exceptions or nested branching.

Composition of Primitives

Basic primitives can be combined into more complex structures. A prompt can be represented as an AST (Abstract Syntax Tree):

Prompt ::= Primitive

| (Prompt $\oplus$ Prompt)
 | Condition(Prompt, Prompt)
 | Label(L, Prompt)
 | Goto(L)

where $\oplus$ is the composition operation of prompts.

Agent Model

An agent is an entity that has a role, maintains an internal state, reacts to events, performs actions, and can communicate with other agents.

Formally, it can be denoted as:

$$a = \langle R, S, L, C \rangle,$$

where:

- R is the role;
- S is the state;
- L is the logic (behavior);
- C is the communication protocol (rules for data exchange).

The structure of an agent can be defined as follows:

```

COMMUNICATION:
  Send_to: {{other_agent_ID}}
TYPE: AGENT
ID: {{unique_name}}
ROLE: {{role_description}}
LOGIC:
  {{Condition, Loop, Function}}
STATE:
  {{key-value pairs, memory, statistics}}
COMMUNICATION:
  Send_to: {{other_agent_ID}}

```

Swarm of Agents

A swarm is a system of interacting agents that simultaneously or sequentially solve parts of a common task.

We formalize this notation as follows:

- $A = \{a_1, a_2, \dots, a_n\}$ – a set of agents;
- T_i – the task of agent a_i ;
- M_{ij} – the transfer of results from $a_i \rightarrow a_j$;
- O_i – the output of agent a_i .

Here is an example of a swarm:

```

TYPE: SWARM
NAME: news_analysis
AGENTS:
  • summarizer_001
  • fact_checker_002
  • sentiment_analyst_003
LOGIC:
  START_PARALLEL([summarizer_001, fact_checker_002])
  IF results_received THEN:
    CALL sentiment_analyst_003 → analyze_results()

```

Parallel Execution via Pseudocode

LLMs do not truly execute actions in parallel, but they can simulate parallelism by explicitly describing separate branches. Therefore, within the framework, a special primitive is introduced:

START_PARALLEL([...])

LOGIC:

```

START_PARALLEL([searcher_001, searcher_002])
MERGE_RESULTS()

```

The formal notation for invoking this primitive is as follows:

$$\text{EXECUTE_PARALLEL}(A) = \{O(a_i)\}_{i=1}^n,$$

where $O(a_i)$ are the results of individual agents' a_i activities.

Agent State (Memory)

An agent's state refers to the history of its past behavior, which is used for adaptation and learning. For example:

STATE:

- fact_base = {...}
- accuracy_statistics = 92%

Formal representation of agent state:

$$S = \{(k_1, v_1), (k_2, v_2), \dots, (k_m, v_m)\}$$

where k_i is a key and v_i is a value.

Inter-Agent Communication

Communication between individual agents can be represented in table form:

PRIMITIVE	DESCRIPTION
Send_message_to (agent_ID)	Specifies the message transmission path
Receive_from(agent_ID)	Waits for a result from another agent
Respond_to(agent_ID)	Sends a response back

Formal definition of a communication act:

$$C(a_i, a_j) = \text{Message}(a_i \rightarrow a_j),$$

where a_i is the sender and, a_j is the receiver.

RESULTS

Ordinary prompts created by a person can be transformed into structured ones according to the proposed framework, also using large language models. For example, by uploading the framework text as a regular document, it is possible to generate structured prompts.

Example of swarm operation in a correlation task

The task is as follows: identify which of the three time series is least correlated with the other two. We proceed to create a structured agent prompt for this purpose:

Let's say you need to calculate mutual correlations between 3 time series, each containing 30 elements, and then identify which series significantly differs from the others. Write an agent-based prompt using this methodology – as a framework.

The input data looks like this:

Data for Analysis:

Series A (currency exchange rate):

[36.5, 36.4, 36.2, 36.3, 36.1, 36.0, 36.1, 36.3, 36.5, 36.7,
36.8, 36.9, 36.8, 36.7, 36.6, 36.5, 36.4, 36.3, 36.2, 36.1,
36.0, 36.1, 36.2, 36.3, 36.4, 36.5, 36.6, 36.7, 36.8, 36.9]

Series B (crypto price):

[28000, 28100, 28200, 28150, 28000, 27900, 27800, 27700, 27800, 27900, 28000, 28100, 28200, 28300, 28400, 28500, 28400, 28300, 28200, 28100, 28000, 27900, 27800, 27700, 27600, 27500, 27400, 27300, 27200, 27100]

Series C (consumer sentiment index):

[50, 51, 50, 49, 48, 47, 48, 49, 50, 51, 52, 53, 54, 53, 52, 51, 50, 49, 48, 47, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55]

As a result of executing this prompt, a structured agent-based prompt is created, which can then be directly input into an LLM:

TYPE: SWARM

NAME: time_series_analysis_and_anomaly_detection

AGENTS:

loader_001

correlator_002

deviation_analyst_003

moderator_004

ROLE: Calculate the correlation matrix between 3 time series and determine which one shows the highest deviation.

LOGIC:

CALL loader_001 → get_series()

IF series_received THEN:

CALL correlator_002 → calculate_correlations(series)

CALL deviation_analyst_003 → detect_anomalous_series(correlation_matrix)

CALL moderator_004 → generate_conclusion(results)

COMMUNICATION:

loader_001 → correlator_002

correlator_002 → deviation_analyst_003

all_results → moderator_004

#Data for Analysis:

Series A (currency exchange rate):

[36.5, 36.4, 36.2, 36.3, 36.1, 36.0, 36.1, 36.3, 36.5, 36.7, 36.8, 36.9, 36.8, 36.7, 36.6, 36.5, 36.4, 36.3, 36.2, 36.1, 36.0, 36.1, 36.2, 36.3, 36.4, 36.5, 36.6, 36.7, 36.8, 36.9]

Series B (crypto price):

[28000, 28100, 28200, 28150, 28000, 27900, 27800, 27700, 27800, 27900, 28000, 28100, 28200, 28300, 28400, 28500, 28400, 28300, 28200, 28100, 28000, 27900, 27800, 27700, 27600, 27500, 27400, 27300, 27200, 27100]

Series C (consumer sentiment index):

[50, 51, 50, 49, 48, 47, 48, 49, 50, 51, 52, 53, 54, 53, 52, 51, 50, 49, 48, 47, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55]

#Agent 1: Loader

TYPE: AGENT

ROLE: Receive time series data for analysis

LOGIC:

```

RECEIVE_INPUT_DATA()
CHECK_FORMAT(each_series_must_be_list_of_30_values)
IF format_valid THEN:
    RETURN_SERIES()
ELSE:
    NOTIFY("Invalid series length")

```

#Agent 2: Correlator

TYPE: AGENT

ROLE: Calculate the correlation matrix between series

LOGIC:

```

RECEIVE_SERIES(from loader_001)
FOR_EACH_PAIR_OF_SERIES:
    CALCULATE_CORRELATION(series_X, series_Y)
CREATE_CORRELATION_MATRIX()

```

FUNCTION calculate_correlation(X, Y):

```

mean_X = sum(X) / len(X)
mean_Y = sum(Y) / len(Y)
numerator = 0
denominator_X = 0
denominator_Y = 0
FOR i FROM 0 TO 29:
    dx = X[i] - mean_X
    dy = Y[i] - mean_Y
    numerator += dx * dy
    denominator_X += dx^2
    denominator_Y += dy^2
RETURN numerator / sqrt(denominator_X * denominator_Y)

```

#Agent 3: Deviation Analyst

TYPE: AGENT

ROLE: Identify which series has the lowest correlation with others

LOGIC:

```

RECEIVE_MATRIX(from correlator_002)
FOR_EACH_SERIES:
    calculate_average_correlation(series_to_others)
FIND_LOWEST_AVERAGE_CORRELATION()
IF below_threshold (e.g., 0.3) THEN:
    mark_as_anomalous()
ELSE:
    normal_series()

```

#Agent 4: Moderator

TYPE: AGENT

ROLE: Provide a structured result

LOGIC:

```

RECEIVE_MATRIX(from correlator_002)
RECEIVE_ANOMALIES(from deviation_analyst_003)
GENERATE_OUTPUT(
    correlation_matrix = {...},
    anomalous_series = [...],
    conclusion = "..."
)

```

EXECUTE:

Read this prompt.

Imagine being each agent in turn:

first loader: check data format and pass it forward,

then correlator: compute pairwise correlations between series,

next deviation analyst: identify which series has the lowest correlation,

finally, moderator: combine everything into a final conclusion.

Do not repeat this prompt – return the result in this format :

```
{
  "correlation_matrix": {
    "A-B": ...,
    "A-C": ...,
    "B-C": ...
  },
  "anomalous_series": [
    {
      "series": "...",
      "description": "...",
      "average_correlation": ...,
      "reason": "..."
    }
  ],
  "conclusion": "..."
}
```

After executing the created structured agent prompt to solve a correlational task, we obtain the following response:

```
{
  "correlation_matrix": {
    "A-B": 0.91,
    "A-C": 0.14,
    "B-C": 0.12
  },
  "anomalous_series": [
    {
      "series": "C",
      "description": "consumer sentiment",
      "average_correlation": 0.13,
      "reason": "weak correlation with exchange rate and crypto activity"
    }
  ],
  "conclusion": "Series C (consumer sentiment) significantly differs from the other two series. Its dynamics are weakly related to the economic indicators of exchange rate and cryptocurrency activity."
}
```

Example of News Analysis

It is required to analyze the news: "The government has approved a new tax law that will affect all citizens."

After applying the AgentFlow framework to the given task, the following structured agent prompt is created:

```
#Prompt for LLM: News Analysis via AgentFlow™ (Single-Stream)
TYPE: SWARM
NAME: news_analysis_swarm
```


AGENTS: loader_001, text_analyst_002, fact_checker_003, emotional_analyst_004, moderator_005

ROLE: Analyze the news article across multiple dimensions: topics, facts, tone, conclusion

LOGIC:

CALL loader_001 → get_text()

IF text_received THEN:

START_PARALLEL([text_analyst_002, fact_checker_003, emotional_analyst_004])

MERGE_RESULTS(to moderator_005)

COMMUNICATION:

loader_001 → text_analyst_002

loader_001 → fact_checker_003

loader_001 → emotional_analyst_004

text_analyst_002 + fact_checker_003 + emotional_analyst_004 → moderator_005

Data for Analysis:

NEWS = "The government has approved a new tax law that will affect all citizens"

#Agent 1: Loader

TYPE: AGENT

ROLE: Accept news input for processing

LOGIC:

RECEIVE_INPUT(NEWS)

CHECK_FORMAT(text)

IF format_valid THEN:

RETURN_DATA()

ELSE:

NOTIFY("Empty/invalid format")

#Agent 2: Text Analyst

TYPE: AGENT

ROLE: Extract keywords and summarize content

LOGIC:

RECEIVE_INPUT(from loader_001)

EXTRACT_KEYWORDS()

GENERATE_SUMMARY(short version)

#Agent 3: Fact Checker

TYPE: AGENT

ROLE: Verify factual accuracy in the text

LOGIC:

RECEIVE_INPUT(from loader_001)

EXTRACT_FACTS()

FOR_EACH fact:

IF fact IN fact_database THEN:

CONFIRM()

ELSE:

FLAG_AS_SUSPICIOUS()

#Agent 4: Emotional Analyst

TYPE: AGENT

ROLE: Determine the tone of the text

```

LOGIC:
  RECEIVE_INPUT(from loader_001)
  DETECT_EMOTIONS()
  IDENTIFY_TONE(positive/neutral/negative)
  PROVIDE_TONE_ASSESSMENT()

#Agent 5: Moderator
TYPE: AGENT
ROLE: Combine results into a single structured output

LOGIC:
  RECEIVE_SUMMARY(from text_analyst_002)
  RECEIVE_FACTS(from fact_checker_003)
  RECEIVE_TONE(from emotional_analyst_004)

  GENERATE_OUTPUT(
    summary = "...",
    facts = [...],
    tone = {...},
    conclusion = "..."
  )

```

Result of executing a structured agent prompt:

```

{
  "summary": "The tax law has been officially approved. The law affects all citizens.",
  "facts": [
    {
      "fact": "the law was approved",
      "status": "officially announced"
    },
    {
      "fact": "applies to all citizens",
      "status": "expected"
    }
  ],
  "emotional_tone": {
    "overall": "neutral",
    "intensity": "low"
  },
  "conclusion": "The tax law has been officially enacted. The news is presented in a neutral manner, without emotional bias."
}

```

DISCUSSION

AgentFlow demonstrates that large language models can serve as a program execution system, where structural primitives in natural language replace traditional syntactic constructs.

LLMs do not execute tasks truly in parallel, but by explicitly describing each branch, they can simulate parallelism.

Each agent acts as an independent module of thought, having its own role, performing a portion of the task, and exchanging results with others. This enables the construction of intelligent systems capable of performing complex tasks through collaborative work.

Agents possess internal memory, which stores past tasks and is added to the agent's experience for future use, shaping new behavior.

AgentFlow is easily adaptable to new tasks, supports automatic agent creation based on simple descriptions, and can be embedded into interfaces for non-technical users.

Conclusions

AgentFlow represents a no-code programming paradigm, where:

- Instead of code – natural language pseudocode is used;
- Instead of syntax – logical primitives are employed;
- Instead of threads – a swarm of agents operates;
- Instead of memory – internal agent states are maintained;
- Instead of interfaces – structured JSON output is produced.

This enables humans to input natural-language queries, obtain structured prompt-agents without writing a single line of code, and receive structured results that can be integrated into other systems.

Conventional prompts for large language models are powerful tools for text generation, answering questions, and even data analysis. However, structured swarm-agent prompts built upon a no-code framework offer significant advantages:

1. Improved logic organization, with clearly defined structural task breakdowns, each aspect of the analysis explicitly separated.
2. Explicit parallel execution via pseudocode, enabling simulation of parallelism – each agent processes its own portion of data independently, leading to deeper, more systematic contextual understanding.
3. Structured output, which allows easy integration of results into other systems such as web interfaces, dashboards, APIs, Notion/Excel documents.
4. Formality, meaning these prompts can be formally described, theoretically analyzed, and deterministically executed – unlike unstructured queries that may yield similar but unpredictable outputs.
5. Agent memory, allowing agents to learn from past experiences and adapt their behavior accordingly.
6. Inter-agent communication, where agents exchange data through explicitly defined interactions, enabling construction of more complex analytical models than simple queries.
7. Scalability and adaptability, allowing these prompts to be easily scaled to N agents, integrated into different tasks, and embedded into user interfaces.
8. Formal execution model, providing a scientific foundation, theoretical description, deterministic execution, and compatibility with formal artificial intelligence models.

Thus, no-code structured swarm-agent prompts enable:

- More precise control over the analysis process;
- Expansion to complex systems;
- Introduction of self-learning mechanisms;
- Seamless integration into other applications.

The scientific novelty lies in the following achievements:

- The first attempt at formalizing a no-code framework using mathematical primitives;
- Introduction of a structured approach implementing the no-code programming paradigm;
- Formal definition of an agent through role, state, logic, and communication;

- Development of a formal agent description usable for theoretical research, automatic agent generation from natural queries, and execution via LLMs as a prompt-execution system;
- Proposal and implementation of a mechanism for parallel process execution through pseudocode, where each agent operates independently and LLM simulates parallelism by explicitly describing branches.

For the first time:

- A formalization of agent internal state was proposed, enabling learning from previous tasks and tracking accuracy statistics for performance improvement.
- A universal output format (JSON-like) was established, facilitating integration into external systems, further analysis, and embedding into various tools.

References

- [1] El Kamouchi, Hind, Mohamed Kissi, and Omar El Beggar. "Low-code/No-code Development: A systematic literature review." In 2023 14th International Conference on Intelligent Systems: Theories and Applications (SITA), pp. 1-8. IEEE, 2023.
- [2] Dmitry Lande, Leonard Strashnoy. An Advanced No-Code Programming Framework for Complex Problems in LLM Environments. Available at ResearchGate DOI: 10.13140/RG.2.2.25307.89129 (May, 2025). – 14 pp.
- [3] Li, Xinyi, Sai Wang, Siqi Zeng, Yu Wu, and Yi Yang. "A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges." Vicinagearth 1, no. 1 (2024): 9.
- [4] Amirkhani, Abdollah, and Amir Hossein Barshooi. "Consensus in multi-agent systems: a review." Artificial Intelligence Review 55, no. 5 (2022): 3897-3935.
- [5] Han, Shanshan, Qifan Zhang, Yuhang Yao, Weizhao Jin, Zhaozhuo Xu, and Chaoyang He. "LLM multi-agent systems: Challenges and open problems." arXiv preprint arXiv:2402.03578 (2024).
- [6] Lande, Dmitry; Strashnoy, Leonard. Implementation Of The Concept Of A "Swarm Of Virtual Experts" In The Formation Of Semantic Networks In The Field Of Cybersecurity Based On Large Language Models. SSRN Preprint: <https://ssrn.com/abstract=4978924>, DOI: 10.2139/ssrn.4978924 (Oct 17, 2024). – 15 p.
- [7] Kim, Sehoon, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. "An llm compiler for parallel function calling." In Forty-first International Conference on Machine Learning. 2024.
- [8] Shakya, Ashish Kumar, Gopinatha Pillai, and Sohom Chakrabarty. "Reinforcement learning algorithms: A brief survey." Expert Systems with Applications 231 (2023): 120495.
- [9] Zhang, Zeyu, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. "A survey on the memory mechanism of large language model based agents." arXiv preprint arXiv:2404.13501 (2024).